

OUAF / ORMB

Complete Technical Study Guide

Artifact Generator • Business Entity • Maintenance Object • Maintenance Extension
Business Component • Business Service • Algorithm • Batches

Framework	Oracle Utilities Application Framework (OUAF)
Product	Oracle Revenue Management and Billing (ORMB)
Audience	Quality Engineers / Technical Consultants / Developers
Prepared	August 2026

All concepts validated against session notes and Oracle OUAF SDK / ORMB documentation.
Use as interview preparation and day-to-day technical reference.

1. Artifact Generator

The **Artifact Generator** is a core tool in the Oracle Utilities Software Development Kit (SDK). It automatically creates the necessary supporting classes required for implementation work.

When you define entities, maintenance objects, or other metadata, the Artifact Generator produces the underlying Java classes the framework expects — POJOs, DTOs, ID classes, generated superclasses, and related support classes.

- Creates all necessary classes for implementation (POJO, supporting DTO classes, ID classes, generated superclasses, etc.).
- Works from metadata defined in the application (tables, MOs, etc.).
- Ensures consistency and reduces manual boilerplate coding.
- Generated classes are the foundation that your hand-written Impl / Extension classes extend.

In practice: after defining metadata, run the Artifact Generator (or use Eclipse wizards) so the Java project has the correct base classes before writing business logic.

2. Business Entity

A **Business Entity** is the Java representation of a persistent table in ORMB / OUAF. When you introduce a new table (or work with an existing one), you must define the corresponding Business Entity in the SDK.

Business entities are transparently initialized and persisted by the framework. They expose useful business methods to callers and contain internal logic (via change handlers) for validation and cascading effects of state changes.

Concept	Description
Business Interface	Public contract (methods exposed to outside callers). Generated onto the entity interface. Coded in the Impl class.
Specialization Interface	Internal / extension-oriented logic (validation, change handling). Not exposed as public API. Handled via change handlers.

Annotation pattern: **@BusinessEntity** (tableName = ..., oneToManyCollections = ...). Tells the framework which table the entity maps to and how child collections are structured.

3. Maintenance Object (MO)

A **Maintenance Object** defines the configuration of a given entity in the system. It includes a primary table and any additional child tables that together capture the physical data for that entity.

MOs provide the structure from which front-end UI components and back-end program components can be generated. Most entities are maintained via their MO (or via Business Objects layered on top of the MO).

How a custom MO is typically created

1. Define metadata in the application (tables, MO definition, related meta-data).
2. In Eclipse / SDK, create the Business Entities for the tables involved.
3. Create the Maintenance Object. In the annotation, declare all the tables / business entities that belong to this MO.
4. The framework / wizard uses that annotation + metadata to generate the maintenance class structure and supporting artifacts.

Note: An MO can also host algorithm plug-in spots. Business Objects (BOs) can be defined on top of an MO to represent different logical types of the same physical entity.

4. Maintenance Extension

A **Maintenance Extension** provides additional functionality to an existing Maintenance Object / Maintenance class. When you need business logic that is not (or cannot be) expressed purely through algorithms, implement it in a Maintenance Extension.

Maintenance extensions must follow the same buffer / structure as the original maintenance. They commonly allow adding new default values and hooking into lifecycle points (after read, after populate element, after change element, etc.).

Annotation pattern: **@MaintenanceExtension** (serviceName = ..., newDefaults = ...). The extension class extends a generated superclass that gives access to structure, header, and default constants.

Use Maintenance Extension when you need to augment the behavior of an MO maintenance program beyond what plug-in algorithms already provide.

5. Business Component

A **Business Component** is a reusable unit of business logic. You write methods that encapsulate common or complex operations, and then invoke those methods from Algorithms, Batches, Maintenance classes, or other components.

This promotes reuse, consistency, and cleaner separation of concerns. Instead of duplicating the same logic inside multiple algorithms or batch workers, centralize it in a Business Component and call it wherever needed.

- Contains reusable methods with business logic.
- Invoked from Algorithms, Batch programs (Worker methods), Maintenance Extensions, etc.
- Keeps Algorithms and Batch code focused on orchestration while the component holds the detailed rules.

In the SDK, Business Components are created as managed classes with their own annotations (e.g. @BusinessComponent).

6. Business Service (Query Page Service)

A **Business Service** (often implemented as a Query Page / Page Maintenance service) is a service that exposes data access and update operations. It is commonly used for query pages and general-purpose page services.

In the annotation you define the service name, the input columns (header / context fields), the output / body columns that need to be populated, and the supported actions (read, change, add, delete, etc.). The framework then routes the call to the corresponding method based on the action.

Supported actions and methods

Action	Typical Method	Purpose
read	read(PageHeader header)	Retrieve data based on input header/criteria and return a DataElement.
change	change(DataElement item)	Apply changes from the input DataElement.
add	add(DataElement item)	Create a new record from the input DataElement and return the result.
delete	delete(DataElement item)	Delete the record represented by the input DataElement.

Annotation (PageMaintenance / QueryPage) declares: service name, body fields (output/data), header fields (input), and the list of supported actions. The framework creates appropriate error handling for unsupported actions.

Related concepts: Page services (often end with P), List services, and Search services form the broader service layer. Business Services can also be invoked from scripts, UI maps, and other components.

7. Algorithm

An **Algorithm** is a specific set of rules / business logic executed at a defined point in the system (a plug-in spot / system event / Algorithm Entity). Algorithms are the primary extension mechanism in OUAF.

Concept	Meaning	
Algorithm Type	Defines the program (Java class or Plug-in Script) and the plug-in spot. Also declares the soft parameters.	
Algorithm	An instance of an Algorithm Type. Multiple algorithms can exist for one type when soft-parameter values differ. The Algorithm is what is plugged in.	
Type	Definition	Source
Hard Parameters	Framework-provided inputs/outputs defined by the plug-in spot. Form the API of that system event.	Defined by the product for each Algorithm Entity / System Event.
Soft Parameters	Configuration values that customize the behavior of a specific algorithm instance.	Declared on Algorithm Type; values supplied on the Algorithm.

Key methods (Java Algorithm Component)

invoke() — Main method where business logic is executed. Framework calls this when the algorithm is triggered.

validateSoftParameters() / **validateExtraSoftParameter...** — Validate soft parameters configured on the Algorithm before it runs in production.

Creation sequence: Program (Java/Script) → Algorithm Type → Algorithm (with values) → Plug into control table / MO / BO.

8. Batches (Background Processes)

A **Batch** is bulk / background processing of a large volume of records. OUAF background processes follow a **Master–Worker** pattern so work can be split across concurrent threads for scalability.

Class	Role	Responsibility
BatchJob (Master)	Master	Determines overall workload, creates JobWork / ThreadWork units, coordinates the job lifecycle.
ThreadWorker (Worker)	Worker	Executes actual business logic for each Work Unit assigned to the thread.

Master Class Methods

getJobWork() — Defines the workload. Write SQL/HQL; results become Work Units. Number of Work Units = load of the batch. Called **once** per job.

finalizeJobWork() — Last method after all threads complete. Post-processing. Called **once** per job.

Worker Class Methods

validateSoftParameters() — Validates soft parameters of the batch. Called once per thread.

initializeThreadWork() — Thread-level setup. Called once per thread.

executeWorkUnit() / **executeWorkUnitDetailResult()** — Actual business logic. Called once per Work Unit.

finalizeThreadWork() — Thread-level cleanup. Called once per thread.

How many times is each method called?

Method	Frequency	Example
getJobWork() / finalizeJobWork()	Once per job	Always 1 time for the whole run
initializeThreadWork() validateSoftParameters() finalizeThreadWork()	Once per thread	If submitted with 5 threads → each called 5 times
executeWorkUnit() / executeWorkUnitDetailResult()	Once per Work Unit	If getJobWork produced 100 Work Units → called 100 times (distributed across threads)

Commit Strategies (ThreadExecutionStrategy)

Strategy	When Commit Happens	Notes / Best Use
Commit Every Unit	After each Work Unit	Many DB hits → performance cost on large volumes. Fine-grained restart.
Standard Commit	After N Work Units (e.g. 30)	Most common. Good balance of throughput and restart granularity.
Single Transaction	Only at the very end	Best for flat-file / extract batches. All-or-nothing.
Thread Iteration	Range-based (Low–High ID) + standard-style commits	Work divided by key ranges. Useful when ranges balance well.

Guidance: Prefer Standard Commit for most transactional jobs. Use Single Transaction for pure extracts. Avoid Commit-Every-Unit on high-volume runs unless there is a strong reason.

Quick Interview Reference

Foundation Layer

Q: What does the Artifact Generator do?

A: Creates necessary supporting classes (POJO, DTO, ID classes, generated superclasses, etc.).

Q: What is a Business Entity?

A: Java representation of a table in ORMB. Must be defined in the SDK.

Q: What is a Maintenance Object?

A: Configuration of an entity: primary table + child tables. Basis for UI and program generation.

Q: How is a custom MO created?

A: Define metadata → create Business Entities → create MO with annotation listing tables/entities → generate artifacts.

Q: What is a Maintenance Extension?

A: Additional functionality added to an MO maintenance class when algorithms alone are not enough.

Q: What is a Business Component?

A: Reusable methods containing business logic that can be invoked from Algorithms, Batches, and other components.

Q: What is a Business Service / Query Page Service?

A: Service that exposes read/change/add/delete operations. Annotation defines input/output columns and actions; methods are called based on the action.

Algorithms

Q: Hard vs Soft parameters?

A: Hard = framework-provided inputs/outputs of the plug-in spot. Soft = configuration values on Algorithm Type / Algorithm.

Q: Where is algorithm business logic written?

A: In the invoke() method of the Algorithm Component.

Q: How do you validate soft parameters?

A: Using validateSoftParameters / validateExtraSoftParameter methods.

Q: Correct creation sequence?

A: Program (Java/Script) → Algorithm Type → Algorithm (with values) → Plug into control table / MO / BO.

Batches

Q: How many main classes in a custom batch?

A: Two: BatchJob (Master) and ThreadWorker (Worker).

Q: Which method defines the workload?

A: getJobWork() in the Master class (called once).

Q: How many times is initializeThreadWork called?

A: Once per thread (e.g. 5 threads → 5 calls).

Q: How many times is executeWorkUnit called?

A: Once per Work Unit (e.g. 100 Work Units → 100 calls, distributed across threads).

Q: Best commit strategy for flat-file extracts?

A: Single Transaction Strategy.

Q: Why avoid Commit-Every-Unit on large volumes?

A: Excessive database commits become a performance bottleneck.

Key Takeaways (All Topics)

- Artifact Generator produces supporting classes (POJO, DTO, ID, generated superclasses).
- Business Entity = Java view of a table; must be defined in the SDK.
- Maintenance Object groups primary + child tables and is the basis for UI and program generation.
- Maintenance Extension adds extra logic to an MO when algorithms are not enough.
- Business Component holds reusable methods that Algorithms and Batches can call.
- Business Service (Query Page) exposes read/change/add/delete; annotation defines inputs/outputs and actions.

- Algorithm = plug-in at a system event; Algorithm Type defines program + soft parameters; Algorithm is the instance.
- Hard parameters come from the plug-in spot; soft parameters are configurable.
- Batch = Master (BatchJob) + Worker (ThreadWorker). `getJobWork` / `finalizeJobWork` = once; thread methods = once per thread; `executeWorkUnit` = once per Work Unit.
- Choose commit strategy carefully — Standard for most jobs, Single Transaction for extracts.

This final study guide consolidates all concepts discussed in the session together with standard Oracle Utilities Application Framework (OUAF) and ORMB documentation patterns. Always cross-check against the specific version and project standards used in your environment.